SIMATS ENGINEERING

ASSESSMENT TOOL – CO4

Theory of Computation (CSA1304)

Industry Problem – Complete Assignment

QUESTION 1 – DFA STRING VALIDATION

Design and implement a C-based validation module that simulates a Deterministic Finite Automaton (DFA) to verify input strings in a real-time system. The module should accept only those strings that begin with the character 'a' and end with the character 'a', ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- **Process user input dynamically.**
- **Validate strings based on DFA state transitions.**
- **Output whether the given input is Accepted or Rejected.**

Answer:

The required language is L = { w ∈ {a,b}* | w begins with a and ends with a }. A DFA can remember whether the first symbol was a and, while reading the rest of the input, remember the most recent symbol. Any string whose first symbol is not a is rejected. The empty string is also rejected.

DFA states:

- q0 – Start state; no input has been read yet.
- q1 – The first symbol was a, and the current last symbol is a. This is the accepting state.
- q2 – The first symbol was a, but the current last symbol is b.
- qd – Dead/reject state; the first symbol was b.

Transition table:

Current State	Input a	Input b
q0	q1	qd

q1	q1	q2
q2	q1	q2
qd	qd	qd

Start state: q0. Accepting state: {q1}.

Example: Input “abba” follows q0 → q1 → q2 → q2 → q1, so it is Accepted. Input “baba” goes q0 → qd and is Rejected.

C program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int state = 0;   // q0

    printf("Enter a string over {a,b}: ");
    scanf("%99s", str);

    for (int i = 0; str[i] != '\0'; i++) {
        char ch = str[i];

        if (state == 0) {
            if (ch == 'a')
                state = 1;
            else
                state = 3;   // dead state
        }
        else if (state == 1) {
            if (ch == 'a')
                state = 1;
            else if (ch == 'b')
                state = 2;
            else
                state = 3;
        }
        else if (state == 2) {
            if (ch == 'a')
                state = 1;
            else if (ch == 'b')
                state = 2;
            else
                state = 3;
        }
        else {
            state = 3;
        }
    }

    if (state == 1)
        printf("Accepted\n");
    else
        printf("Rejected\n");

    return 0;
}
```

Sample output:

```
Enter a string over {a,b}: abba
Accepted
```

```
Enter a string over {a,b}: bab
Rejected
```

QUESTION 2 – PUSH DOWN AUTOMATA

Design a Push Down Automata that accepts the language described in the question, where the number of 0's in w is equal to the number of 1's in w.

Answer:

The language is interpreted as $L = \{ w \in \{0,1\}^* \mid \#0(w) = \#1(w) \}$. A PDA can accept this language by using its stack to cancel a 0 against an unmatched 1 and a 1 against an unmatched 0. This works irrespective of the order of 0s and 1s.

PDA idea:

- If the next input is 0 and the top of the stack is Z0 (bottom symbol) or 0, push 0; if the top is 1, pop 1.
- If the next input is 1 and the top is Z0 or 1, push 1; if the top is 0, pop 0.
- After the complete input is consumed, accept when only the bottom-of-stack symbol Z0 remains.
- The stack therefore stores the unmatched excess symbols. The stack becomes empty above Z0 exactly when the numbers of 0s and 1s are equal.

Formal PDA:

$M = (Q, \Sigma, \Gamma, \delta, q0, Z0, F)$, where $Q = \{q\}$, $\Sigma = \{0,1\}$, $\Gamma = \{0,1,Z0\}$, start state $q0 = q$, initial stack symbol Z0, and $F = \{qf\}$. The transition behavior is given below.

Input	Top of Stack	Action
0	Z0	Push 0
0	0	Push 0
0	1	Pop 1
1	Z0	Push 1
1	1	Push 1
1	0	Pop 0

Acceptance: when the input is completely read and the stack contains only Z0, the PDA enters qf and accepts.

Example for w = 0011: Z0 → 0Z0 → 00Z0 → 0Z0 → Z0, so the string is Accepted. For w = 001, one 0 remains unmatched, so the PDA rejects.

QUESTION 3 – PDA FOR L = {$a^n b^{2n}$ | n ≥ 1}

Design a Pushdown Automata for the language representing a's followed by b's, where the number of b's must be twice the number of a's: L = {a^n b^2n | n ≥ 1}.

Answer:

The PDA first reads all a's and pushes two stack markers for every a. After the first b is seen, it changes to a b-processing state and pops one marker for every b. Thus, for n a's, exactly 2n markers are placed on the stack, requiring exactly 2n b's to remove them.

States:

- q0 – Start state; requires at least one a.
- q1 – Reads a's and pushes two X symbols for every a.
- q2 – Reads b's and pops one X for every b.
- qf – Accepting state when all X symbols have been removed and input is exhausted.

Transition description:

State	Input	Stack top	Operation / Next state
q0	a	Z0	Push XXZ0; go to q1
q1	a	X	Push XXX (net effect: add XX); stay q1
q1	b	X	Pop X; go to q2
q2	b	X	Pop X; stay q2
q2	ε	Z0	Go to qf

Note: In the q1 transition, the notation is expressed as a stack replacement to show that two additional X markers are added while preserving the existing X. An equivalent implementation can use two ε-transitions to push the two markers separately.

Example: For n = 2, input aabb should be accepted. Each a contributes two X markers, so four X markers are pushed. The four b's pop them one by one, leaving Z0, and the PDA accepts.

Strings such as aab, aaabbbb, or baabb are rejected because they do not satisfy the required order and/or the exact 2:1 ratio.

QUESTION 4 – ε-CLOSURE OF NFA STATES

Develop a C-based computational module to determine the ε-closure of all states in a Non-Deterministic Finite Automaton (NFA) with ε-transitions, as part of an automata processing engine used in compiler construction and pattern-matching systems.

Answer:

The ε-closure of a state q is the set of states reachable from q using zero or more ε-transitions. Therefore, q itself is always included in ε-closure(q). To compute it, a stack is initialized with q. Whenever a state is removed from the stack, every state reachable through an ε-transition is added if it has not already been visited.

The following C program accepts the number of states and an ε-transition matrix and prints the ε-closure of every state. In the matrix, 1 means an ε-transition exists and 0 means it does not.

```c
#include <stdio.h>

#define MAX 20

int n;
int eps[MAX][MAX];
int visited[MAX];
int stack[MAX];

void epsilonClosure(int start) {
    int top = -1;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[start] = 1;
    stack[++top] = start;

    while (top >= 0) {
        int current = stack[top--];

        for (int next = 0; next < n; next++) {
            if (eps[current][next] == 1 && !visited[next]) {
                visited[next] = 1;
                stack[++top] = next;
            }
        }
    }

    printf("Epsilon-closure(q%d) = { ", start);
    for (int i = 0; i < n; i++) {
        if (visited[i])
            printf("q%d ", i);
    }
    printf("}\n");
}

int main() {
    printf("Enter number of states: ");
    scanf("%d", &n);

    printf("Enter epsilon-transition matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &eps[i][j]);

    printf("\nEpsilon closures:\n");
    for (int i = 0; i < n; i++)
        epsilonClosure(i);

    return 0;
}
```

Sample input:

```
Enter number of states: 4
Enter epsilon-transition matrix:
0 1 0 0
0 0 1 0
0 0 0 0
0 0 0 1
```

Sample output:

```
Epsilon closures:
Epsilon-closure(q0) = { q0 q1 q2 }
Epsilon-closure(q1) = { q1 q2 }
Epsilon-closure(q2) = { q2 }
Epsilon-closure(q3) = { q3 }
```

The algorithm is suitable for an automata-processing module because it systematically explores all ε-reachable states and prevents repeated visits using the visited array.

QUESTION 5 – CFG-BASED BINARY STRING VALIDATION

Develop a C-based input validation module that determines whether a given binary string conforms to the specified Context-Free Grammar (CFG): S → 0A1, A → 0A | 1A | ε.

Answer:

From the grammar, S always produces one 0 at the beginning and one 1 at the end. The nonterminal A can generate any sequence of 0s and 1s, including the empty string. Therefore, the language generated by the grammar is L = {0w1 | w ∈ {0,1}*}. In other words, a valid string must begin with 0, end with 1, and have any binary sequence in between.

Examples:

- 01 → S ⇒ 0A1 ⇒ 01, Accepted.
- 001 → S ⇒ 0A1 ⇒ 00A1 ⇒ 001, Accepted.
- 0111 → S ⇒ 0A1 ⇒ 01A1 ⇒ 0111, Accepted.
- 101 → Rejected because it does not begin with 0.
- 010 → Rejected because it does not end with 1.

C program:

```c
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int valid = 1;

    printf("Enter a binary string: ");
    scanf("%99s", str);

    int len = strlen(str);

    if (len < 2) {
        valid = 0;
    } else if (str[0] != '0' || str[len - 1] != '1') {
        valid = 0;
    } else {
        for (int i = 0; i < len; i++) {
            if (str[i] != '0' && str[i] != '1') {
                valid = 0;
                break;
            }
        }
    }

    if (valid)
        printf("Accepted: String conforms to the CFG.\n");
    else
        printf("Rejected: String does not conform to the CFG.\n");

    return 0;
}
```

Sample output:

```
Enter a binary string: 010101
Accepted: String conforms to the CFG.
```

```
Enter a binary string: 10101
Rejected: String does not conform to the CFG.
```